prd: latency-instrumented hinglish voice agent

entry for the gnani.ai great indian ai internship challenge. deadline: nov 10, 2026, 11:59 pm ist. this doc is scoped for a same-day build. anything not marked as measured is a target or an assumption, not a result.

1. summary

a voice agent that answers scholarship and fee status questions in hinglish over synthetic student data. the product is secondary. the point of the project is the pipeline instrumentation: every request logs per-stage latency (asr, llm, tts, time-to-first-audio), and the same benchmark is run on a sequential baseline and a streamed/overlapped version so the optimization is measured, not claimed.

gnani models used: prisma (streaming speech-to-text), timbre (text-to-speech, beta). evon is not used (not on the api platform, and too large for local 6gb vram).

non-gnani components: one hosted llm with tool calling (whichever key is already available), python, fastapi, a minimal browser mic page.

2. problem and user

user: a college student who would rather ask "meri scholarship ka status kya hai?" out loud than navigate a portal.

problem: voice agents feel slow when stages run one after another. users hear dead air between finishing a sentence and hearing a reply. most demos hide this.

what this project shows: where the time actually goes, and how much streaming and chunked tts recover, with real numbers and stated limitations.

3. goals and non-goals

goals

- working end-to-end loop: speech in, tool-backed answer, speech out
- per-stage latency logging on every request
- a/b benchmark: sequential baseline vs streamed pipeline, same inputs

- honest writeup including failures

non-goals

- real student data, real auth, real portal integration
- multi-turn dialogue beyond one clarification turn
- production deployment, scaling, or a polished ui
- claiming accuracy numbers from a tiny test set as general performance

4. scope: the one narrow task

status lookup for two intents:

1. scholarship status (applied / under review / approved / rejected, amount, next step)
2. fee status (paid / due amount / due date)

example utterances (hinglish):

- "meri scholarship approve hui ya nahi?"
- "fees kitni baaki hai?"
- "mera roll number S1004 hai, scholarship ka status batao"

identification: spoken roll number from a synthetic set (e.g. `S1001` to `S1010`). agent reads the number back for confirmation before lookup. this is a demo mechanism, not real authentication, and the writeup should say so.

5. architecture

```
mic / wav file
   |
   v
[capture + endpointing]  -> t_end_of_speech
   |
   v
[prisma streaming asr]   -> t_asr_first_partial, t_asr_final
   |
   v
[llm + tool calls]       -> t_llm_request, t_tool_start/end,
t_llm_first_token, t_llm_first_sentence
   |
   v
[sentence chunker]
   |
   v
[timbre tts, per chunk]  -> t_tts_request, t_tts_first_byte
   |
   v
[playback queue]         -> t_first_audio_out
```

two modes, same code path, one flag:

- **baseline (sequential):** wait for final transcript, wait for full llm response, send full text to tts, play after full audio returns.
- **streamed (overlapped):** stream audio to prisma while the user speaks, start the llm on final transcript, split output into sentence chunks, send the first chunk to tts as soon as it is complete, play while later chunks synthesize.

adapters. wrap prisma and timbre behind two small interfaces (`transcribe_stream(audio) -> events`, `synthesize(text) -> audio bytes/stream`) so the rest of the code does not depend on api details. check docs.gnani.ai for the actual endpoints, auth, audio format and streaming protocol before writing the adapters. none of those details are assumed in this doc.

6. functional requirements

id	requirement	priority
f1	accept audio input from a wav file (benchmark mode)	must
f2	stream audio to prisma and receive partial and final transcripts	must
f3	llm decides intent and calls `get_scholarship_status(roll_no)` or `get_fee_status(roll_no)`	must
f4	tools read from a synthetic json or sqlite file	must
f5	reply is generated in hinglish and synthesized by timbre	must
f6	every request writes one jsonl record with all stage timestamps	must
f7	baseline and streamed modes selectable by flag	must
f8	benchmark script runs n utterances x modes and writes a results csv	must
f9	browser page with mic button for live demo	should
f10	summary script prints p50 / p95 per stage per mode	should
f11	if roll number is missing or unconfirmed, agent asks once instead of guessing	should
f12	noisy-audio variant of the test set	could

7. latency metrics

all timestamps taken with a monotonic clock on one machine. define each term once and use it consistently in the writeup.

- **asr_latency:** `t_asr_final - t_end_of_speech` (how long after the user stops until the final transcript exists)

- **llm_ttft:** `t_llm_first_token - t_llm_request`
- **tool_latency:** `t_tool_end - t_tool_start`
- **llm_first_sentence:** `t_llm_first_sentence - t_llm_request` (what the streamed mode actually waits for)
- **tts_ttfb:** `t_tts_first_byte - t_tts_request` (for the first chunk)
- **time_to_first_audio (ttfa):** `t_first_audio_out - t_end_of_speech` (the headline number)
- **total_response_time:** `t_last_audio_end - t_end_of_speech`

report p50 and p95, plus n. with a small n, say so and show the raw distribution rather than only a summary.

in benchmark mode with pre-recorded wav files, `t_end_of_speech` is the moment the last audio chunk is sent to prisma (the file has no real-time endpointing). note this in the writeup so the number is not confused with live-mic latency.

8. targets and hypotheses

these are hypotheses to test, not results. fill in the real numbers after the benchmark runs.

- h1: streamed mode has lower ttfa than baseline on every utterance
- h2: the largest share of baseline ttfa is llm generation plus full-response tts, not asr
- h3: first-sentence chunking cuts ttfa mainly by removing the wait for the full llm response

if a hypothesis turns out false, report that. it is a better story than a faked win.

9. benchmark design

- **test set:** 20 utterances minimum. mix of scholarship and fee intents, with and without roll number in the same sentence, some hinglish code-mixing, a few pure hindi and pure english. recorded by you (and a consenting friend if possible). no real personal data.
- **conditions:** clean room, and one noisy condition (fan, traffic or market audio played in the background, or noise mixed in digitally).
- **runs:** each utterance x each mode, 3 repeats to see variance. run baseline and streamed interleaved, not all baseline first, so network drift does not favor one mode.

- **correctness check (small):** for each utterance, record whether the transcript was usable and whether the tool call and final answer were right. report as counts, e.g. "x of 20", not as a general accuracy claim.
- **record:** machine, network type, region, date and time of runs, model versions as reported by the apis.

10. synthetic data

10 fake students in `students.json`:

```
{
  "S1004": {
    "name": "synthetic student",
    "scholarship": {"status": "under_review", "amount": 25000, "next_step":
"document verification"},
    "fees": {"due": 12000, "due_date": "2026-12-05"}
  }
}
```

vary statuses across all four scholarship states and include one student with zero dues. state clearly in the submission that all data is synthetic, as the challenge rules require.

11. risks

risk	mitigation
prisma misrecognizes spoken digits in roll numbers	read the number back and confirm; log the failure rate and report it
timbre is beta; chunk boundaries may sound choppy	measure and describe it honestly; try sentence-level vs clause-level chunking only if time remains
api rate limits or credit exhaustion (5,000 credits)	cache nothing in benchmark runs, but budget calls up front: 20 utterances x 2 modes x 3 repeats x 2 conditions is 240 runs, so check credit cost per call first and cut repeats if needed

risk	mitigation
streaming protocol details differ from what is assumed	read docs.gnani.ai first; ask in the challenge discord if blocked
same-day deadline	build order in section 12; benchmark harness before the browser demo
llm adds variance unrelated to the voice stack	fix temperature, keep the system prompt short, note the llm used

12. build order (same-day)

1. **adapters first.** get one prisma call and one timbre call working in isolation with a wav in and a wav out.
2. **baseline pipeline.** sequential path end to end with timestamps and jsonl logging.
3. **tools and synthetic data.** two tools, llm tool-calling loop, hinglish system prompt.
4. **streamed pipeline.** sentence chunker, per-chunk tts, playback queue. keep the baseline untouched for comparison.
5. **benchmark script and summary.** run both modes on the recorded set.
6. **record, write, post.** only after real numbers exist.
7. **browser mic demo** only if time remains. a screen recording of the cli plus a latency printout is an acceptable demo.

if something has to be cut, cut in this order: browser page, noisy condition, repeats. do not cut the benchmark.

13. submission checklist

- [ ] registered, api key working
- [ ] gnani models named in the submission: prisma, timbre
- [ ] working demo video, about 60 seconds, opening with the problem and the latency printout, not the stack
- [ ] written explanation: who it is for, what it does, models used, measured results, limitations, what to fix next
- [ ] synthetic-data statement

- [ ] linkedin or x post with #GreatIndianAIInternshipChallenge and #GnaniAI, posted and nominated before the deadline, not on the last minute
- [ ] programme terms on the registration page read, especially ownership
- [ ] repo public, readme with real current state only

14. writeup outline

1. problem and user, two sentences
2. architecture, one diagram
3. what was measured and how (definitions from section 7)
4. results table: baseline vs streamed, p50 / p95 per stage, n stated
5. what the numbers say, including anything that did not improve
6. failures: digit recognition, noisy condition, chunk boundary artifacts, whatever actually happened
7. what i would change with more time